Report primo progetto

INTRODUZIONE AL PROGETTO

Il progetto volge a dimostrare la velocità della programmazione in parallelo rispetto ad una programmazione in serie.

COSA FA IL PROGRAMMA

CREAZIONE DEI DATI

Il programma genera un segnale periodico con rumore "bianco" (all'interno del file **iniz.cpp**).

```
for(int i = 0; i < grand_vet; i++){
    vet[i] = (amp * sin(i/periodo)) + aggiunta_rumore(gen, amp);
}
```

Per riproducibilità è stato fissato un seed di generazione, quindi il rumore sarà sempre lo stesso.

```
mt19937 gen(11);
```

Nello specifico è stata usata una distribuzione uniforme dalla libreria random per creare il rumore (all'interno del file **iniz.cpp**).

```
double ampiezza_rumore = amp / 5.0;
uniform_real_distribution<double> dist(-ampiezza_rumore, ampiezza_rumore);
return dist(gen);
```

CALCOLO SMA

In seguito è stata calcolata una SMA(Simple Moving Avarage) che consiste nel fare la media degli ultimi tot numeri decisi dall'utente (il parmaetro all'interno del programma è **WINDOW**). Questo passo è quello che è stato parallelizzato, all'interno del file: **indicator.cppp** (per **OpenMP**) e **cuda.cu** (per **CUDA**). Il risultato è un segnale con un rumore minore, ma con un offset rispetto a quello iniziale.

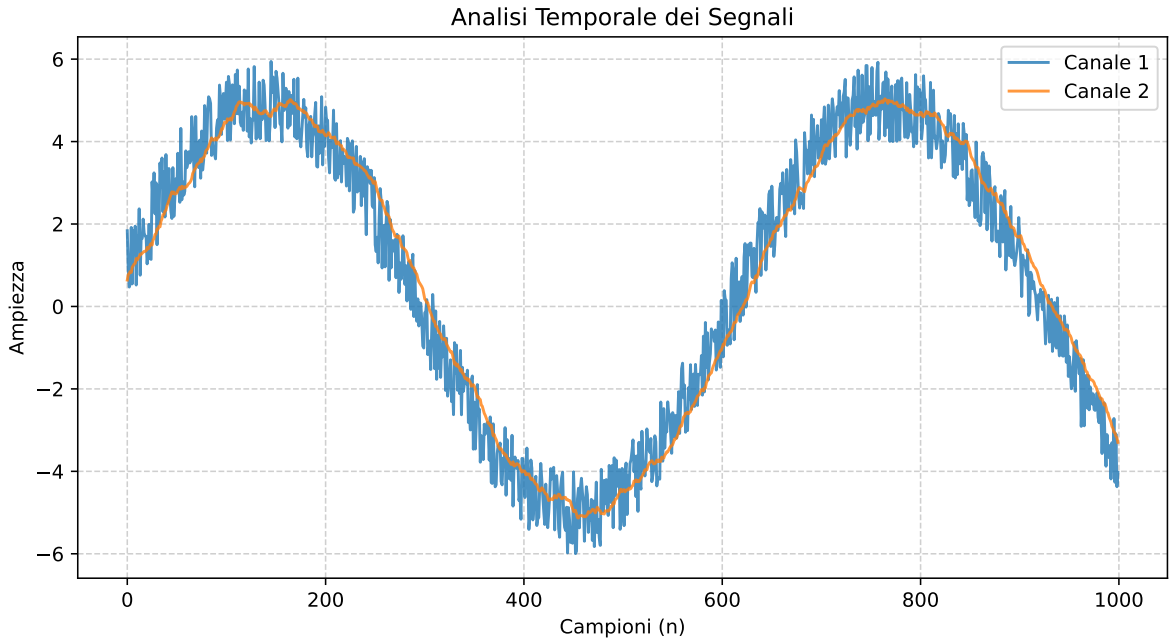


Figure 1: Confronto dei due segnali

COSA SERVE UN SMA

Un ambito conosciuto è il trading, dove con l'aiuto di questo strumento si può diminuire il rumore dei mercati finanziari, aiutando la comprensione di questi.

COME È CALCOLATA

Come accennato sopra è stato fatto in modi diversi. Precisamente sono stati utilizzati tre metodi, due con **OpenMP** e uno per **CUDA**.

Ciclo esterno (OpenMP)

Nel **Makefile** è stato nominato **outer**, indica che la parallelizzazione è stata fatta sul ciclo esterno quindi quello più grande.

```
double start = omp_get_wtime();

#pragma omp parallel for schedule(runtime)
for(int i = 0; i < len; i++){
    double sum = 0;
    int index = i + window - 1;
    for(int j = index; j >= i; j--){
        sum += vet[j];
    }
    sma[i] = sum / window;
}
double stop = omp_get_wtime();
return stop - start;
```

Ciclo interno (OpenMP)

Nel **Makefile** è stato nominato **intern**, indica che la parallelizzazione è stata fatta sul ciclo interno quindi quello più piccolo.

```
double start = omp_get_wtime();

for(int i = 0; i < len; i++){
    double sum = 0;
    int index = i + window - 1;

    #pragma omp parallel for reduction(+:sum) schedule(runtime)
    for(int j = index; j >= i; j--){
        sum += vet[j];
    }

    sma[i] = sum / window;
}

double stop = omp_get_wtime();
return stop - start;
```

CUDA

Nel **Makefile** è stato nominato **CUDA**.

```
int i = blockIdx.x * blockDim.x + threadIdx.x;

if (i < len) {
    double sum = 0.0;
    int index_fine = i + window - 1;
    for (int j = index_fine; j >= i; j--) {
        sum += vet[j];
    }
    sma[i] = sum / (double)window;
}
```

DATI RACCOLTI

I dati sono stati inseriti in un file denominato **dati.csv** e riportati sottostante.

Table 1: Confronto performance (tempi in secondi)

tipo	threads	grand_vet	window	num_cili_per_rep	reps	tempo_medio	tempo_min	tempo_m
outer	1	524288	256	134676224	100	0.261133	0.256509	0.269513
outer	1	2097152	256	538902272	100	1.046730	1.032380	1.083080
outer	1	2097152	512	1075576320	100	2.185270	2.158700	2.332210
intern	1	2097152	512	1075576320	100	2.634520	2.604850	2.713520
outer	2	2097152	512	1075576320	100	1.104160	1.085220	1.138670
outer	2	2097152	512	1075576320	100	1.149920	1.135630	1.187710
outer	2	2097152	512	1075576320	100	1.104180	1.091780	1.139160
intern	2	2097152	512	1075576320	50	2.349320	2.310980	2.405920
intern	2	2097152	512	1075576320	10	21.391700	21.197900	21.572000
intern	2	2097152	512	1075576320	50	3.321900	3.261130	3.395220
outer	4	2097152	512	1075576320	100	0.569013	0.563615	0.577725
outer	4	2097152	512	1075576320	100	0.608953	0.603293	0.617365
outer	4	2097152	512	1075576320	100	0.567488	0.562190	0.575012
intern	4	2097152	512	1075576320	50	2.759770	2.706010	3.023220
intern	4	2097152	512	1075576320	10	20.437700	20.314500	20.543200
intern	4	2097152	512	1075576320	25	5.573270	5.522620	5.621740
outer	8	2097152	512	1075576320	100	0.297995	0.294780	0.304634
outer	8	2097152	512	1075576320	100	0.337350	0.318163	0.350494
outer	8	2097152	512	1075576320	100	0.320876	0.316344	0.328322
intern	8	2097152	512	1075576320	25	3.855000	3.776120	3.908230
intern	8	2097152	512	1075576320	25	26.014800	25.625100	26.351100
intern	8	2097152	512	1075576320	25	12.569100	12.257500	12.911900
outer	16	2097152	512	1075576320	100	0.222198	0.192713	0.276007

tipo	threads	grand_vet	window	num_cili_per_rep	reps	tempo_medio	tempo_min	tempo_m
outer	16	2097152	512	1075576320	100	0.245060	0.230294	0.263215
outer	16	2097152	512	1075576320	100	0.234931	0.206041	0.280611
outer	32	2097152	512	1075576320	100	0.233839	0.220801	0.257740
outer	32	2097152	512	1075576320	100	0.240515	0.226554	0.255617
outer	32	2097152	512	1075576320	100	0.239917	0.213624	0.265097
outer	64	2097152	512	1075576320	100	0.228945	0.214186	0.250227
outer	64	2097152	512	1075576320	100	0.240993	0.228735	0.261100
outer	64	2097152	512	1075576320	100	0.226540	0.216726	0.266667
outer	128	2097152	512	1075576320	100	0.226116	0.216069	0.253758
outer	128	2097152	512	1075576320	100	0.240734	0.230708	0.269435
outer	128	2097152	512	1075576320	100	0.224267	0.216348	0.288819
CUDA	32	2097152	512	1075576320	100	0.007154	0.007043	0.007624
CUDA	64	2097152	512	1075576320	100	0.007141	0.007000	0.007484
CUDA	128	2097152	512	1075576320	100	0.007145	0.007016	0.007473
CUDA	256	2097152	512	1075576320	100	0.007131	0.007009	0.009521
CUDA	512	2097152	512	1075576320	100	0.007117	0.006994	0.007405
CUDA	1024	2097152	512	1075576320	100	0.007139	0.007033	0.007537
CUDA	32	33554432	512	17213160960	100	0.112278	0.112058	0.118636
CUDA	64	33554432	512	17213160960	100	0.111844	0.111661	0.112337
CUDA	128	33554432	512	17213160960	100	0.112337	0.111636	0.118920
CUDA	256	33554432	512	17213160960	100	0.112149	0.111462	0.112659
CUDA	512	33554432	512	17213160960	100	0.111523	0.111298	0.111814
CUDA	1024	33554432	512	17213160960	100	0.112662	0.111964	0.119839

Tabella dati

SPEED-UP

Lo **Speed-up** è calcolato come:

$$S_{up} = \frac{S_{base}}{S_{calc}}$$

Indica quanto veloce è laparallelizzazione rispetto al programma eseguito sequenzialmente.

Ciclo esterno

Table 2: Speed-up delle implementazioni Outer (solo test con carico di lavoro coerente)

	Tipo	Threads	Tempo Medio (s)	Speed-up
2	outer	1	2.1853	1.00
4	outer	2	1.1042	1.98
5	outer	2	1.1499	1.90
6	outer	2	1.1042	1.98
10	outer	4	0.5690	3.84
11	outer	4	0.6090	3.59
12	outer	4	0.5675	3.85
16	outer	8	0.2980	7.33
17	outer	8	0.3374	6.48
18	outer	8	0.3209	6.81
22	outer	16	0.2222	9.83
23	outer	16	0.2451	8.92
24	outer	16	0.2349	9.30
25	outer	32	0.2338	9.35
26	outer	32	0.2405	9.09
27	outer	32	0.2399	9.11
28	outer	64	0.2289	9.54
29	outer	64	0.2410	9.07
30	outer	64	0.2265	9.65
31	outer	128	0.2261	9.66
32	outer	128	0.2407	9.08
33	outer	128	0.2243	9.74

Da questa tabella si nota che il test migliore ha otttenuto uno **Speed-up** di **9.83**.

Inizialmente lo Speed-up è circa due volte superiore a quello del numero di Thread inferiore quindi un andamento lineare.

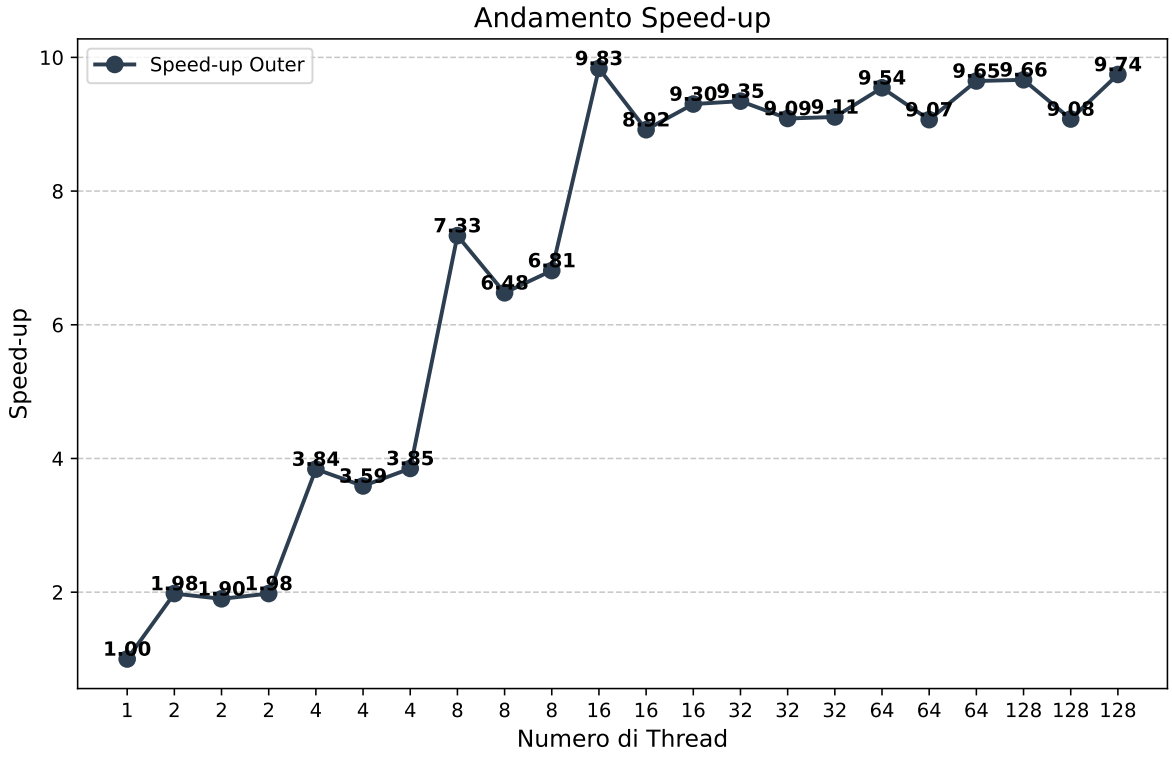


Figure 2: Speed-up Outer con distribuzione uniforme sull'asse X